

MÁSTER EN INTELIGENCIA ARTIFICIAL

VERIFICACIÓN DOCUMENTAL
DE IDENTIDAD Y SOLICITUDES
DE PRÉSTAMO MEDIANTE
VISIÓN ARTIFICIAL Y DEEP
LEARNING

TFM elaborado por: Raúl Hernando Viadero

Tutor/a de TFM: Juan Manuel Moreno Lamparero

- Madrid, junio de 2026 -

Resumen

Palabras clave: visión artificial, aprendizaje profundo, verificación documental, YOLOv8, EasyOCR, ResNet-18, DNI, préstamos, datos sintéticos, RGPD.

El presente Trabajo de Fin de Máster propone y desarrolla un sistema end-to-end de verificación documental automatizada para entidades financieras, capaz de procesar expedientes de solicitud de préstamo formados por el Documento Nacional de Identidad (DNI) del solicitante y el formulario de solicitud cumplimentado.

El sistema integra tres componentes de inteligencia artificial: detección de regiones de interés mediante YOLOv8n con mAP@0.5 del 99,5%; extracción de texto mediante EasyOCR con Field Accuracy superior al 82% en todos los campos; y clasificación de autenticidad mediante ResNet-18 con fine-tuning, alcanzando un AUC-ROC del 93,6%. Sobre los campos extraídos se aplican nueve reglas de validación cruzada que verifican la coherencia entre DNI y formulario, la vigencia documental y la viabilidad financiera del préstamo. El sistema completo alcanza una precisión del 94% en la clasificación final del veredicto.

El dataset de entrenamiento es 100% sintético, generado programáticamente con Faker (locale es_ES) y Pillow, garantizando el pleno cumplimiento del RGPD al no utilizarse en ningún momento datos personales reales. El sistema se despliega mediante Docker Compose con una API REST (FastAPI, puerto 8000) y una interfaz web (Streamlit, puerto 8501), y se acompaña de una suite de 77 tests automatizados.

Abstract

Keywords: computer vision, deep learning, document verification, YOLOv8, EasyOCR, ResNet-18, Spanish ID card, loans, synthetic data, GDPR.

This Master's Thesis proposes and develops an end-to-end automated document verification system for financial institutions, processing loan application dossiers consisting of the applicant's Spanish National Identity Document (DNI) and the completed application form.

The system integrates three artificial intelligence components: region of interest detection using YOLOv8n (mAP@0.5 = 99.5%); text extraction using EasyOCR (Field Accuracy > 82% across all

fields); and authenticity classification using fine-tuned ResNet-18 (AUC-ROC = 93.6%). Nine cross-validation rules verify consistency between the DNI and form data, document validity and financial feasibility, achieving 94% precision in the final verdict. All training data is 100% synthetic (Faker, es_ES locale), ensuring full GDPR compliance. The system is deployed via Docker Compose with a REST API (FastAPI) and a web interface (Streamlit), backed by a suite of 77 automated tests.

Índice

Resumen.....	iii
Abstract.....	iii
Índice.....	iv
Índice de tablas.....	vi
1. Introducción y Antecedentes.....	1
1.1. Contexto y motivación.....	1
1.2. Antecedentes técnicos.....	2
1.2.1. Detección de objetos con redes neuronales.....	2
1.2.2. Reconocimiento óptico de caracteres.....	3
1.2.3. Clasificación de autenticidad documental.....	3
1.2.4. Privacidad y datos sintéticos.....	4
1.3. Descripción del problema.....	4
1.4. Estructura de la memoria.....	5
2. Objetivos del Proyecto.....	6
2.1. Objetivo general.....	6
2.2. Objetivos específicos.....	6
3. Material y Métodos.....	7
3.1. Entorno de desarrollo.....	7
3.2. Stack tecnológico.....	7
3.3. Generación del dataset sintético.....	8
3.3.1. Diseño del generador de datos.....	8
3.3.2. Generación de imágenes sintéticas.....	9
3.3.3. Augmentación de datos.....	10
3.3.4. Partición del dataset.....	10
3.4. Modelos de inteligencia artificial.....	11
3.4.1. YOLOv8n — Detección de regiones de interés.....	11
3.4.2. ResNet-18 — Clasificador de autenticidad.....	11
3.4.3. Postprocesador de texto OCR.....	12
3.5. Motor de validación cruzada.....	13
3.6. Pipeline de inferencia end-to-end.....	13
3.7. Arquitectura del sistema desplegado.....	14
3.8. Verificación y validación del software.....	15
4. Resultados.....	16
4.1. Dataset generado.....	16
4.2. Análisis exploratorio del dataset.....	16
4.3. Resultados de los modelos YOLO.....	16

4.4. Resultados del clasificador de autenticidad.....	17
4.5. Evaluación del pipeline OCR.....	18
4.6. Evaluación end-to-end.....	19
4.7. Demostración del sistema: verificación paso a paso.....	20
4.8. Ejemplos de uso del sistema.....	21
4.8.1. Uso desde la interfaz web (analista de riesgos).....	21
4.8.2. Uso desde la API REST (integración de sistemas).....	21
4.8.3. Uso programático (analista de datos).....	22
5. Conclusiones.....	23
5.1. Conclusiones principales.....	23
5.2. Limitaciones.....	24
5.3. Líneas de trabajo futuro.....	25
6. Referencias Bibliográficas.....	26
7. Anexos.....	28
Anexo A. Material entregado y estructura del repositorio.....	28
Anexo B. Reglas de validación cruzada.....	29
Anexo C. Instrucciones de despliegue.....	29
Anexo D. Ejemplo de expediente inconsistente.....	30

Índice de tablas

Tabla 1. Stack tecnológico del sistema de verificación documental.

Tabla 2. Partición del dataset sintético (400 expedientes, 15% inconsistentes, seed=42).

Tabla 3. Ejemplos de normalización aplicada por el postprocesador de texto.

Tabla 4. Métricas de los modelos YOLOv8n en el split de test.

Tabla 5. Métricas del clasificador ResNet-18 en el split de test.

Tabla 6. Field Accuracy y confianza OCR media por campo en el split de test.

Tabla 7. Métricas del sistema end-to-end en el split de test (50 expedientes).

Tabla A1. Descripción de las nueve reglas de validación cruzada.

1. Introducción y Antecedentes

1.1. Contexto y motivación

La digitalización del sector financiero ha transformado radicalmente los procesos de concesión de crédito. Las entidades bancarias reciben diariamente miles de solicitudes de préstamo que requieren la verificación manual de documentos de identidad y formularios de datos personales y financieros, un proceso costoso, lento y propenso a errores humanos. Según estimaciones del sector, un analista experimentado dedica entre 10 y 20 minutos a verificar manualmente un expediente completo, comprobando la coherencia entre los datos del documento de identidad y los declarados en el formulario, la vigencia del documento y la viabilidad financiera de la operación.

La verificación manual de documentos presenta tres limitaciones principales. En primer lugar, un alto coste operativo derivado del tiempo dedicado por analistas especializados, que escala linealmente con el volumen de solicitudes. En segundo lugar, inconsistencia en la aplicación de criterios de validación entre diferentes revisores: dos analistas pueden emitir veredictos distintos sobre el mismo expediente en función de su experiencia o del cansancio acumulado. Y en tercer lugar, vulnerabilidad frente a documentos falsificados o datos manipulados, especialmente con la creciente sofisticación de las herramientas de edición digital, que hacen que las alteraciones sean imperceptibles a simple vista.

A estas limitaciones operativas se suma un contexto regulatorio cada vez más exigente. Los procesos de alta de clientes y concesión de crédito están sujetos a las obligaciones de diligencia debida conocidas como KYC (Know Your Customer) y a la normativa de prevención del blanqueo de capitales, que en España desarrolla la Ley 10/2010 y que obligan a las entidades a verificar de forma fehaciente la identidad de sus clientes. Al mismo tiempo, la banca compite por ofrecer procesos de contratación 100% digitales en los que el cliente fotografía sus documentos desde el móvil y espera una respuesta en minutos. Conciliar la exigencia regulatoria con la inmediatez que demanda el mercado es precisamente el hueco que ocupan los sistemas automáticos de verificación documental.

La visión artificial y el aprendizaje profundo ofrecen una solución a estas limitaciones mediante la automatización del proceso de verificación documental. Los detectores de objetos basados en redes neuronales convolucionales permiten localizar con precisión los campos de texto en documentos de alta resolución, los motores de reconocimiento óptico de caracteres (OCR)

modernos alcanzan tasas de error muy bajas incluso en condiciones adversas de captura, y los clasificadores de imagen pueden detectar patrones de manipulación invisibles para el ojo humano. La combinación de estas tres tecnologías con un motor de reglas de negocio permite construir un sistema de cribado automático que procesa cada expediente en segundos, aplica siempre los mismos criterios y deja constancia trazable de cada decisión.

Conviene subrayar el papel que juega cada tecnología en esa cadena y por qué ninguna es suficiente por sí sola. El detector de objetos aporta estructura: convierte una fotografía en un conjunto de regiones tipadas (esto es el nombre, esto es el NIF), pero no lee su contenido. El OCR aporta el contenido, pero no sabe qué significa ni si es coherente. El clasificador de autenticidad examina la integridad visual del soporte, pero es ciego al significado de los datos. Y el motor de reglas, que es donde reside el conocimiento del negocio, solo puede operar si las tres etapas anteriores le entregan datos fiables y estructurados. Esta complementariedad es la tesis arquitectónica central del trabajo: la fiabilidad del sistema no descansa en un único modelo, sino en el contraste sistemático entre fuentes de evidencia independientes.

1.2. Antecedentes técnicos

1.2.1. Detección de objetos con redes neuronales

La detección de objetos es la tarea de visión artificial que consiste en localizar (mediante un rectángulo delimitador o bounding box) y clasificar simultáneamente los objetos presentes en una imagen. La familia de modelos YOLO (You Only Look Once), introducida por Redmon et al. (2016), revolucionó la detección de objetos en tiempo real al formular el problema como una única regresión desde los píxeles de la imagen hasta las coordenadas de los bounding boxes y las probabilidades de clase, en lugar del enfoque en dos etapas (propuesta de regiones + clasificación) de arquitecturas anteriores como R-CNN.

La versión YOLOv8, desarrollada por Ultralytics (2023), introduce mejoras arquitectónicas significativas: una cabeza de detección desacoplada y libre de anclas (anchor-free), el bloque C2f que mejora el flujo de gradiente respecto al C3 de YOLOv5, y la función de pérdida Distribution Focal Loss para la regresión de cajas. La variante nano (YOLOv8n), con solo 3,2 millones de parámetros, ofrece un equilibrio excelente entre precisión y velocidad de inferencia, lo que la hace idónea para entornos sin GPU dedicada como el de este proyecto. En el contexto de documentos de identidad, la detección de campos es un problema especialmente favorable para YOLO: los documentos tienen una estructura espacial fija (el nombre siempre está en la misma zona relativa), lo que permite alcanzar precisiones muy altas con datasets moderados.

1.2.2. Reconocimiento óptico de caracteres

El reconocimiento óptico de caracteres (OCR) transforma imágenes de texto en cadenas de caracteres procesables. Los motores OCR modernos combinan una etapa de detección de texto (localizar dónde hay texto en la imagen) con una etapa de reconocimiento (transcribir cada región detectada). EasyOCR (Jaided AI, 2020) es un motor de código abierto que utiliza la red CRAFT (Baek et al., 2019) para la detección y una arquitectura CRNN —red convolucional + LSTM bidireccional + decodificación CTC— (Shi et al., 2017) para el reconocimiento, con soporte para más de 80 idiomas, incluido el español.

A diferencia de sistemas clásicos como Tesseract, EasyOCR no requiere binarización previa de la imagen y es robusto frente a variaciones de iluminación, ruido y pequeñas rotaciones, características esenciales para el procesamiento de documentos escaneados o fotografiados con dispositivos móviles. No obstante, ningún motor OCR es infalible: confusiones típicas como $O \leftrightarrow 0$, $I \leftrightarrow 1$, $S \leftrightarrow 5$ o $B \leftrightarrow 8$, la fusión de caracteres adyacentes y los errores en símbolos poco frecuentes (€, /) obligan a incorporar una etapa de postprocesamiento que normalice y corrija el texto extraído en función del tipo de campo esperado (fecha, importe, NIF...). Este postprocesamiento contextual es una de las contribuciones centrales del presente trabajo.

1.2.3. Clasificación de autenticidad documental

La detección de documentos manipulados mediante redes neuronales convolucionales ha sido abordada por numerosos trabajos en la literatura de image forensics. Las manipulaciones más habituales (sustitución de la fotografía, alteración de campos de texto, recompresión de regiones editadas) dejan artefactos estadísticos en la imagen — discontinuidades en el ruido del sensor, dobles compresiones JPEG, bordes anómalos — que una red convolucional puede aprender a detectar aunque sean invisibles para un observador humano.

El enfoque de fine-tuning sobre arquitecturas preentrenadas en ImageNet, como ResNet-18 (He et al., 2016), ha demostrado ser especialmente efectivo cuando el volumen de datos de entrenamiento es limitado. La intuición es que las capas convolucionales inferiores de una red preentrenada ya han aprendido representaciones visuales de bajo nivel (bordes, texturas, gradientes) que son transferibles a cualquier dominio de imagen; por tanto, basta con reentrenar las capas superiores para especializar el modelo en la nueva tarea, reduciendo drásticamente el riesgo de sobreajuste y el tiempo de entrenamiento. ResNet-18 introduce además las conexiones residuales (skip connections), que permiten entrenar redes profundas sin degradación del gradiente.

1.2.4. Privacidad y datos sintéticos

El Reglamento General de Protección de Datos (RGPD, Reglamento (UE) 2016/679) impone restricciones estrictas al tratamiento de datos personales, y los documentos de identidad constituyen datos especialmente sensibles. Esto dificulta enormemente la creación de datasets de documentos reales para investigación: se requeriría el consentimiento explícito e informado de cada titular, medidas técnicas de seudonimización y una base jurídica sólida para el tratamiento.

El uso de datos sintéticos generados programáticamente — enmarcado en las técnicas de Privacy-Preserving AI (Kaissis et al., 2020) — elimina este problema de raíz: si ningún dato corresponde a una persona real, el RGPD no resulta de aplicación. La clave del enfoque es que los datos sintéticos deben ser estructuralmente idénticos a los reales: los NIF deben tener letras de control válidas, las zonas MRZ deben cumplir el estándar ICAO 9303, las cuotas de los préstamos deben ser matemáticamente coherentes con el importe, plazo e interés. Solo así los modelos entrenados sobre datos sintéticos generalizan al dominio real. Este trabajo demuestra que la síntesis rigurosa de documentos es un sustituto viable de los datos reales para tareas de verificación documental.

1.3. Descripción del problema

El presente trabajo aborda la automatización del proceso de verificación de expedientes de solicitud de préstamo en una entidad financiera española ficticia (Banco Digital Español). Un expediente completo consta de dos documentos:

- **El Documento Nacional de Identidad (DNI)** del solicitante (anverso), que contiene nueve regiones de información: nombre, apellidos, número de documento con letra de control, fecha de nacimiento, fecha de caducidad, nacionalidad, fotografía, firma y zona de lectura mecánica (MRZ) de dos líneas.
- **El formulario de solicitud de préstamo** cumplimentado, con catorce campos organizados en cuatro secciones: datos personales del solicitante (nombre, apellidos, NIF, fecha de nacimiento, teléfono, email), datos laborales (situación laboral, ingresos netos mensuales, antigüedad), y condiciones del préstamo (importe solicitado, plazo en meses, finalidad, cuota mensual resultante y TIN aplicado).

La verificación automática del expediente requiere resolver cinco subproblemas encadenados: (a) detectar y recortar los campos de texto de ambos documentos; (b) transcribir el texto de cada recorte y normalizarlo para corregir los errores sistemáticos del OCR; (c) verificar que los datos del formulario coinciden con los del DNI (mismo titular); (d) comprobar que el documento está vigente y que el solicitante cumple los requisitos de capacidad jurídica (mayoría de edad) y financiera (ratio de endeudamiento); y (e) clasificar ambos documentos como legítimos o potencialmente manipulados. La salida del sistema es un veredicto binario (APTO PARA TRÁMITE / INCONSISTENTE) acompañado de una puntuación de confianza y del detalle de cada comprobación, de modo que un analista humano pueda revisar únicamente los expedientes marcados como sospechosos.

1.4. Estructura de la memoria

El resto de la memoria se organiza como sigue. El capítulo 2 enumera el objetivo general y los objetivos específicos del proyecto. El capítulo 3 describe el material y los métodos: el entorno de desarrollo, el stack tecnológico, el diseño del generador de datos sintéticos, la arquitectura y entrenamiento de los modelos, el pipeline de inferencia y la arquitectura de despliegue. El capítulo 4 presenta y discute los resultados obtenidos por cada componente y por el sistema completo, incluyendo un ejemplo de verificación paso a paso. El capítulo 5 recoge las conclusiones, limitaciones y líneas de trabajo futuro. El capítulo 6 lista las referencias bibliográficas en formato APA y el capítulo 7 contiene los anexos técnicos.

2. Objetivos del Proyecto

2.1. Objetivo general

Diseñar, implementar y evaluar un sistema end-to-end de verificación documental automatizada para expedientes de solicitud de préstamo que integre técnicas de visión artificial y aprendizaje profundo para la detección de regiones de interés, extracción de texto, clasificación de autenticidad y validación de reglas de negocio, y que se despliegue como servicio web reproducible mediante contenedores.

2.2. Objetivos específicos

1. Generar un dataset sintético de documentos DNI y formularios de préstamo, 100% libre de datos personales reales, estructuralmente fiel a los documentos reales (NIF válidos, MRZ ICAO 9303, cuotas coherentes) y suficiente para el entrenamiento y evaluación de los modelos.
2. Entrenar dos modelos YOLOv8n para la detección de los 9 campos del DNI y los 14 del formulario, con el objetivo de alcanzar un mAP@0.5 superior al 85%.
3. Implementar un pipeline de extracción y normalización de texto que corrija los errores de OCR más frecuentes en documentos españoles (confusiones de caracteres, formatos de fecha e importe, símbolos monetarios).
4. Entrenar un clasificador binario de autenticidad documental basado en ResNet-18 con fine-tuning, con el objetivo de alcanzar un AUC-ROC superior al 90%.
5. Implementar un motor de validación cruzada con nueve reglas de negocio (R01–R09) que compruebe la coherencia entre los datos del DNI y del formulario, la vigencia documental y la viabilidad financiera.
6. Desarrollar una API REST con FastAPI que exponga el pipeline completo como servicio web documentado (OpenAPI/Swagger).
7. Implementar una interfaz web con Streamlit para subir documentos, visualizar el veredicto y el detalle de cada comprobación, y exportar informes PDF.
8. Contenerizar el sistema completo con Docker Compose para garantizar un despliegue reproducible en cualquier entorno.
9. Verificar la calidad del software mediante una suite de tests automatizados con pytest.

3. Material y Métodos

3.1. Entorno de desarrollo

El proyecto se ha desarrollado íntegramente en un entorno local con las siguientes características: sistema operativo Windows 11 con WSL2 (Windows Subsystem for Linux 2) para la ejecución de contenedores; lenguaje Python 3.11 gestionado con Anaconda; control de versiones Git con repositorio público en GitHub (https://github.com/RaulHernandoViadero/validacion_prestamos); y contenerización con Docker Desktop. El flujo de trabajo siguió la convención de ramas main (estable) y dev (desarrollo), con commits atómicos siguiendo la especificación Conventional Commits (feat:, fix:, test:, docs:).

El entrenamiento de los modelos se realizó exclusivamente en CPU, sin GPU dedicada. Esta restricción, lejos de ser una limitación menor, condicionó decisiones de diseño relevantes: la elección de la variante nano de YOLOv8 (la más ligera de la familia), el fine-tuning parcial de ResNet-18 (congelando el 75% de los parámetros) y la resolución de entrada de 640×640 píxeles para YOLO. El resultado demuestra que es posible construir un sistema de verificación documental funcional sin infraestructura especializada.

3.2. Stack tecnológico

La Tabla 1 recoge las tecnologías utilizadas y su función en el sistema. La selección priorizó herramientas de código abierto, maduras y con comunidades activas:

Categoría	Tecnología	Versión
Detección de objetos	Ultralytics YOLOv8n	8.3.39
OCR	EasyOCR	1.7.2
Deep Learning	PyTorch + torchvision	2.5.1 / 0.20.1
Generación de datos	Pillow + Faker	11.0.0 / 33.1.0
API REST	FastAPI + Uvicorn	0.115.5 / 0.32.1
Frontend	Streamlit	1.41.1
Informes PDF	ReportLab	4.2.5
Modelos de datos	Pydantic v2	2.10.2
Contenedores	Docker Compose	v2
Testing	pytest	8.3.4

Tabla 1. Stack tecnológico del sistema de verificación documental.

Merece la pena justificar brevemente las decisiones tecnológicas frente a sus alternativas más habituales. Se eligió EasyOCR frente a Tesseract porque no requiere binarización previa de la imagen y su arquitectura neuronal es notablemente más robusta ante variaciones de iluminación, ruido y pequeñas rotaciones, exactamente el tipo de degradaciones que introduce la augmentación de datos. FastAPI se prefirió frente a Flask por tres motivos: la validación automática de entradas y salidas mediante esquemas Pydantic (que elimina toda una familia de errores en los contratos de la API), la generación automática de documentación interactiva OpenAPI/Swagger y su soporte nativo de operaciones asíncronas. Streamlit se eligió frente a Dash o una aplicación web en JavaScript por la velocidad de desarrollo que ofrece para prototipos analíticos. Por último, PyTorch se utilizó como framework de deep learning por ser el sustrato común tanto de Ultralytics como de EasyOCR, lo que unifica la gestión de dependencias del proyecto.

3.3. Generación del dataset sintético

3.3.1. Diseño del generador de datos

El módulo `fake_data_factory.py` genera perfiles de solicitante completos y coherentes usando Faker con la localización `es_ES`. La coherencia estructural es el requisito clave: cada dato sintético debe superar las mismas validaciones que superaría un dato real. Los cuatro mecanismos principales son:

a) NIF con letra de control válida. El número de DNI español consta de 8 dígitos y una letra de control calculada como el resto de dividir el número entre 23, usado como índice sobre la cadena oficial de letras:

```
LETRAS = "TRWAGMYFPDXBNJZSQVHLCKE"
letra = LETRAS[numero % 23]

# Ejemplo: 77898694 % 23 = 17 → LETRAS[17] = "V" → NIF: 77898694V
```

Esta misma fórmula se reutiliza en la regla de validación R03 para verificar que el NIF extraído por OCR es aritméticamente válido: si el OCR confunde un dígito, la letra de control deja de coincidir y el error se detecta automáticamente.

b) Zona MRZ según ICAO 9303. La zona de lectura mecánica del DNI sigue el formato TD1 del estándar ICAO 9303, con líneas de 30 caracteres y dígitos de control calculados con el algoritmo de pesos cíclicos 7-3-1: cada carácter se convierte a un valor numérico (dígitos = su valor; letras A-Z = 10-35; relleno < = 0), se multiplica por el peso correspondiente de la secuencia 7,3,1,7,3,1..., y el dígito de control es la suma módulo 10:

```
# Dígito de control del número de documento "77898694":
# carácter:  7   7   8   9   8   6   9   4
# peso:      7   3   1   7   3   1   7   3
# producto: 49  21   8  63  24   6  63  12   → suma = 246
# dígito de control = 246 mod 10 = 6
```

El sistema recalcula estos dígitos de control durante la verificación (comprobación mrz_checkdigits_ok), lo que aporta una vía independiente para detectar tanto errores de OCR como manipulaciones de la zona MRZ.

c) Datos financieros coherentes. La cuota mensual del préstamo se calcula con la fórmula de amortización francesa (sistema de cuota constante, el estándar en la banca española):

$$C = P \cdot r \cdot (1 + r)^n / ((1 + r)^n - 1)$$

donde C es la cuota mensual, P el capital prestado, r el tipo de interés mensual (TIN anual / 12) y n el número de mensualidades. Por ejemplo, para un préstamo de 42.500 € a 84 meses con un TIN anual del 6% (r = 0,005): $(1,005)^{84} = 1,5203$, y por tanto $C = 42.500 \cdot 0,005 \cdot 1,5203 / 0,5203 \approx 621$ €/mes. La regla R07 del validador reutiliza esta fórmula para comprobar que la cuota declarada en el formulario es coherente con el importe, plazo e interés declarados.

d) Fechas coherentes. Las fechas de nacimiento se generan en el rango 1950–2000 (garantizando la mayoría de edad), y la fecha de caducidad del DNI se fija 10 años después de la de emisión, conforme a la normativa española para mayores de 30 años. Un subconjunto controlado de documentos se genera deliberadamente caducado para poblar los casos negativos de la regla R05.

3.3.2. Generación de imágenes sintéticas

Las imágenes se renderizan con Pillow a alta resolución. El DNI se genera a 1518×957 píxeles (proporción de la tarjeta ID-1 a escala ×3), con la disposición visual del DNI 3.0 español: fotografía a la izquierda, campos rotulados en la zona central-derecha, firma manuscrita simulada mediante curvas Bézier aleatorias, y zona MRZ inferior renderizada con tipografía OCR-

B. El formulario se genera a 1588×2246 píxeles (A4 a 96 dpi, escala ×2) con un diseño corporativo bancario en cuatro secciones diferenciadas con cabeceras de color, campos con etiqueta gris y valor negro, y una tabla resumen de las condiciones del préstamo.

Junto a cada imagen se emite su anotación en formato YOLO: un fichero de texto con una línea por campo, con la clase y el bounding box normalizado (x_centro, y_centro, anchura, altura en fracción de la imagen). Al generarse imagen y anotación en el mismo proceso, las cajas son exactas por construcción — no existe ruido de etiquetado humano, lo que explica en parte las métricas muy altas alcanzadas por los detectores.

3.3.3. Augmentación de datos

Para que los modelos generalicen más allá de las imágenes limpias, cada imagen del split de entrenamiento se replica en 3 variantes aleatorias que simulan condiciones reales de captura: rotaciones de $\pm 5^\circ$, transformación de perspectiva leve (simulando fotografía no perpendicular), ruido gaussiano, variaciones de brillo y contraste ($\pm 25\%$), desenfoque suave y oclusiones parciales rectangulares (simulando dedos o reflejos). Las anotaciones YOLO se transforman geométricamente junto a la imagen para mantener la correspondencia exacta de las cajas.

Para el clasificador de autenticidad se genera adicionalmente una versión manipulada de cada documento, aplicando una de cuatro alteraciones aleatorias: sustitución de la fotografía por otra, edición de un campo de texto (parche del color de fondo + texto distinto), recompresión JPEG agresiva de una región (simulando un pegado digital) y clonación de regiones. Estas manipulaciones reproducen los vectores de fraude documental más habituales.

3.3.4. Partición del dataset

Se generaron 400 expedientes completos (400 DNIs + 400 formularios) con semilla aleatoria fija (seed=42) para garantizar la reproducibilidad. El 15% de los expedientes contiene inconsistencias deliberadas (datos que no coinciden entre DNI y formulario, documentos caducados, ratios de endeudamiento excesivos) para poblar la clase negativa del veredicto. La partición se realizó a nivel de expediente — nunca de imagen — para evitar fugas de información entre splits:

Split	Expedientes	Porcentaje	Imágenes tras augmentación
Entrenamiento	280	70%	1.101 por tipo de documento
Validación	70	17,5%	70 por tipo de documento

Test	50	12,5%	50 por tipo de documento
------	----	-------	--------------------------

Tabla 2. Partición del dataset sintético (400 expedientes, 15% inconsistentes, seed=42).

3.4. Modelos de inteligencia artificial

3.4.1. YOLOv8n — Detección de regiones de interés

La familia YOLOv8 se distribuye en cinco variantes de tamaño creciente (n, s, m, l, x) que comparten arquitectura y se diferencian en anchura y profundidad de la red: la variante nano tiene 3,2 millones de parámetros, la small 11,2 y la medium 25,9. En un benchmark preliminar sobre un subconjunto de 50 imágenes, la variante small mejoraba el mAP@0.5:0.95 en menos de un punto porcentual respecto a la nano, a costa de triplicar el tiempo de entrenamiento e inferencia en CPU. Dado que la nano ya superaba con holgura los objetivos del proyecto, la relación coste-beneficio de las variantes mayores no se justificaba: se seleccionó YOLOv8n para ambos detectores.

Se entrenaron dos modelos YOLOv8n independientes — uno por tipo de documento — partiendo de los pesos preentrenados en COCO. Entrenar modelos separados en lugar de uno conjunto simplifica el espacio de clases de cada modelo y permite optimizarlos de forma independiente. Las 9 clases del modelo de DNI son: nombre, apellidos, numero_dni, fecha_nacimiento, fecha_caducidad, nacionalidad, foto, firma y mrz_line. Las 14 clases del modelo de formulario cubren los datos personales (sol_nombre, sol_apellidos, sol_nif, sol_fecha_nacimiento, sol_telefono, sol_email), laborales (sol_situacion_laboral, sol_ingresos_netos, sol_antiguedad) y del préstamo (prestamo_importe, prestamo_plazo, prestamo_finalidad, prestamo_cuota, prestamo_tin).

La configuración de entrenamiento fue: máximo de 100 épocas con parada temprana (patience=20), batch de 16 imágenes, resolución de entrada 640×640, optimizador AdamW con learning rate inicial de 0,01 y los hiperparámetros de augmentación interna de Ultralytics ajustados al dominio documental — en particular, fliplr=0 y flipud=0, puesto que un documento con texto nunca aparece especularmente invertido y permitir esos volteos degradaría el aprendizaje.

3.4.2. ResNet-18 — Clasificador de autenticidad

El clasificador de autenticidad es una ResNet-18 preentrenada en ImageNet con fine-tuning parcial: las capas conv1, bn1, layer1, layer2 y layer3 permanecen congeladas (conservando las

representaciones visuales genéricas aprendidas en ImageNet) y solo se entrenan layer4 y la capa de clasificación fc, lo que supone 8,4 de los 11,2 millones de parámetros totales. La capa fc original (1000 clases de ImageNet) se sustituye por Dropout(0,3) seguido de Linear(512, 2) para la clasificación binaria LEGÍTIMO / MANIPULADO.

La configuración de entrenamiento fue: 30 épocas, optimizador AdamW con learning rate de 1e-4, scheduler CosineAnnealingLR, función de pérdida CrossEntropyLoss y batch de 32. El conjunto de entrenamiento contiene 3.320 imágenes balanceadas (1.660 legítimas y 1.660 manipuladas), con normalización según las estadísticas de ImageNet y redimensionado a 224×224 píxeles.

3.4.3. Postprocesador de texto OCR

El módulo TextPostprocessor aplica normalizaciones específicas según el tipo semántico de cada campo, corrigiendo los errores sistemáticos de EasyOCR. Algunos ejemplos reales de transformaciones aplicadas:

Campo	Texto OCR crudo	Texto normalizado	Corrección aplicada
numero_dni	77898694 v	77898694V	Eliminación de espacios, mayúsculas
fecha_nacimiento	15 03 1985	15/03/1985	Reconstrucción de separadores
prestamo_importe	27,182.13 €	27182.13	Formato anglosajón detectado
prestamo_importe	334.631 €lmes	334.63	'€/mes' leído como '€lmes'
sol_ingresos_netos	2.500	2500.00	Separador de miles europeo
nombre	JACOBO	JACOBO	Confusión 0↔O en campo alfabético

Tabla 3. Ejemplos de normalización aplicada por el postprocesador de texto.

La normalización de importes es el caso más delicado: la cadena "27,182.13" debe interpretarse como formato anglosajón (27182,13 €) mientras que "334.631" con tres decimales aparentes es en realidad un separador de miles europeo. El postprocesador resuelve la ambigüedad analizando la posición y número de dígitos tras cada separador. En los campos alfabéticos

(nombres, apellidos) se aplican sustituciones inversas de las confusiones numéricas (0→O, 1→I, 5→S) y se eliminan los caracteres no alfabéticos residuales.

3.5. Motor de validación cruzada

El motor de validación ejecuta nueve reglas de negocio (R01–R09) sobre los campos normalizados de ambos documentos. Cada regla emite un resultado estructurado con código, nombre, resultado (pasada/fallida), severidad (ERROR bloquea el veredicto; WARNING solo advierte) y un detalle textual explicativo. Las nueve reglas se describen íntegramente en el Anexo B; a modo de ejemplo, la regla R07 (ratio de endeudamiento) opera así:

```
# R07 – Ratio de endeudamiento ≤ 35%
ratio = cuota_mensual / ingresos_netos

# Ejemplo con el expediente EXP00009:
#   cuota = 621.00 €/mes ; ingresos = 2.500 €/mes
#   ratio = 621 / 2500 = 0.248 → 24,8% ≤ 35% → R07 PASADA
```

Las comparaciones de texto entre documentos (R01, R02) no exigen igualdad estricta, sino una similitud de Levenshtein normalizada igual o superior al 85%. Este umbral tolera errores residuales de OCR de uno o dos caracteres en apellidos largos sin dejar pasar nombres realmente distintos: por ejemplo, "LOBO AGUDO" frente a "LOBO AGUDO" (con un cero) tiene una similitud del 90% y pasa la regla, mientras que "LOBO AGUDO" frente a "LOPEZ AGUDO" cae por debajo del umbral y la regla falla.

El veredicto final se emite con una puntuación de confianza que combina la proporción de reglas superadas con la calidad media del OCR:

$$\text{confianza} = 0,70 \cdot (\text{reglas_ok} / 9) + 0,30 \cdot \text{confianza_OCR_media}$$

La ponderación 70/30 refleja que la coherencia de los datos es el criterio dominante, pero que una extracción OCR de baja calidad debe rebajar la confianza aunque las reglas pasen (los datos comparados podrían ser erróneos). Un expediente es APTO PARA TRÁMITE si todas las reglas de severidad ERROR pasan; en caso contrario es INCONSISTENTE y se listan las reglas fallidas.

3.6. Pipeline de inferencia end-to-end

El orquestador DocumentPipeline encadena las seis etapas del proceso y constituye el punto de entrada único del sistema:

10. Detección YOLO: los dos modelos localizan los ROIs en el DNI y el formulario, devolviendo los recortes con un margen de contexto del 8% de la altura de cada caja.

11. Extracción OCR: EasyOCR transcribe cada recorte (excluyendo foto y firma). Si un ROI devuelve texto vacío y su caja empieza lejos del borde izquierdo, se aplica un mecanismo de reserva que relee la misma franja horizontal completa desde $x=0$, recuperando campos cuyo texto quedó fuera de la caja detectada.
12. Normalización: el TextPostprocessor corrige y tipifica el texto de cada campo según su tipo semántico.
13. Clasificación de autenticidad: ResNet-18 clasifica cada documento completo como LEGÍTIMO o MANIPULADO con su probabilidad.
14. Validación cruzada: se ejecutan las reglas R01–R09 sobre los campos normalizados.
15. Veredicto: se calcula la confianza global y se construye la respuesta estructurada con el detalle completo de cada etapa.

El pipeline es tolerante a fallos por diseño: cada etapa está aislada de modo que una excepción (un modelo no disponible, un campo ilegible) degrada el resultado — añadiendo una advertencia y marcando las reglas afectadas como no evaluables — pero nunca detiene el proceso completo. Esta propiedad es esencial en un sistema de cribado: es preferible emitir un veredicto INCONSISTENTE con advertencias que no emitir ninguno.

3.7. Arquitectura del sistema desplegado

El sistema se despliega con Docker Compose en dos servicios que se comunican por la red interna `verificacion-net`:

- **verificacion-api** (puerto 8000): imagen multi-stage sobre `python:3.11-slim` que contiene FastAPI, los tres modelos y el pipeline completo. La construcción multi-stage separa la compilación de dependencias del contenedor final, y el servicio se ejecuta con un usuario sin privilegios (`appuser`) siguiendo las buenas prácticas de seguridad en contenedores.
- **verificacion-frontend** (puerto 8501): interfaz Streamlit que consume la API. Declara una dependencia con condición de salud (`healthcheck`) sobre el servicio api, de modo que solo arranca cuando la API responde correctamente al endpoint `/health`.

La API expone los endpoints principales POST /api/v1/verify (verificación completa de un expediente, multipart con las dos imágenes), GET /api/v1/history (historial de expedientes de la sesión), GET /api/v1/metrics (estadísticas agregadas) y GET /health (estado del servicio y de los modelos). Un ejemplo abreviado de la respuesta JSON de verificación:

```
{
  "expediente_id": "EXP00009",
  "resultado": "APTO PARA TRAMITE",
  "es_apto": true,
  "confianza": 0.961,
  "validacion": {
    "n_reglas_ok": 9, "n_reglas_fallo": 0,
    "reglas": [ {"codigo": "R01", "nombre": "Coincidencia nombre",
      "pasada": true, "detalle": "JACOBO = JACOBO (100%)"}, ... ]
  },
  "autenticidad_dni": {"etiqueta": "LEGITIMO", "confianza": 0.968},
  "autenticidad_form": {"etiqueta": "LEGITIMO", "confianza": 0.957},
  "perfil_riesgo": {"nivel": "BAJO", "ratio_deuda": 0.248},
  "tiempo_total_ms": 8842
}
```

La interfaz Streamlit organiza la aplicación en cuatro páginas: Verificar Expediente (subida de documentos, veredicto con semáforo visual, detalle por pestañas de reglas, campos OCR, perfil de riesgo y autenticidad, y exportación del informe en PDF), Historial (expedientes procesados en la sesión), Métricas (KPIs del servicio) y Dashboard (distribución de veredictos y análisis de fallos por regla).

3.8. Verificación y validación del software

La calidad del código se verificó mediante una suite de 77 tests automatizados con pytest, organizados en tres bloques: tests del generador de datos sintéticos (validez de NIFs, dígitos de control MRZ, coherencia de cuotas), tests unitarios del pipeline (postprocesador de texto, reglas de validación individuales con casos positivos y negativos, cálculo del veredicto) y tests de integración de la API (endpoints con imágenes reales de test, códigos de error, validación de esquemas Pydantic). Los 77 tests pasan en la versión final del sistema, y la suite se ejecuta también durante la construcción de la imagen Docker.

4. Resultados

4.1. Dataset generado

Se generaron 800 imágenes de alta resolución (400 DNIs de 1518×957 px y unos 190 KB de media, y 400 formularios de 1588×2246 px y unos 220 KB) con sus 9.200 anotaciones YOLO correspondientes (3.600 cajas de DNI y 5.600 de formulario). Tras la augmentación, el conjunto de entrenamiento asciende a 1.101 imágenes por tipo de documento. El 85% de los expedientes son consistentes y el 15% contiene discrepancias intencionales que simulan los escenarios de fraude o error administrativo.

La generación completa del dataset (datos, imágenes, augmentación, anotaciones y manipulaciones para el clasificador de autenticidad) tarda aproximadamente 25 minutos en el equipo de desarrollo y es totalmente reproducible: la semilla aleatoria fija garantiza que cualquier evaluador que ejecute `generate_dataset.py` con los mismos parámetros obtenga exactamente los mismos 400 expedientes, con los mismos nombres, NIFs, importes e inconsistencias. Esta reproducibilidad determinista es una ventaja adicional del enfoque sintético frente a los datasets reales: los resultados experimentales de este trabajo pueden replicarse íntegramente desde cero.

4.2. Análisis exploratorio del dataset

El análisis exploratorio (notebook `01_data_exploration.ipynb`) confirma que las distribuciones del dataset son realistas y aptas para el entrenamiento: importe medio del préstamo de 42.500 € (rango 1.000–100.000 €), plazo medio de 84 meses, ingresos netos entre 1.200 y 4.800 €/mes (media 2.500 €/mes) y ratio de endeudamiento medio del 22%, con los expedientes inconsistentes concentrados por encima del límite del 35%. Las clases del clasificador de autenticidad están perfectamente balanceadas (50% legítimos / 50% manipulados), lo que evita sesgos de clase en el entrenamiento. La verificación automática de la coherencia interna del dataset (letras de NIF, dígitos MRZ, cuotas francesas) arroja un 100% de validez, confirmando la corrección del generador.

4.3. Resultados de los modelos YOLO

La Tabla 4 resume las métricas de ambos detectores sobre el split de test. `mAP@0.5` es la precisión media considerando correcta una detección cuyo solapamiento (IoU) con la caja real

supera el 50%; $mAP@0.5:0.95$ promedia sobre umbrales de IoU crecientes de 0,5 a 0,95 y es por tanto mucho más exigente con la precisión de la localización:

Modelo	$mAP@0.5$	$mAP@0.5:0.95$	Precisión	Recall
YOLOv8n DNI (9 clases)	99,5%	98,9%	99,9%	100,0%
YOLOv8n Formulario (14 clases)	99,5%	95,5%	99,7%	100,0%

Tabla 4. Métricas de los modelos YOLOv8n en el split de test.

Ambos modelos superan ampliamente el objetivo fijado ($mAP@0.5 > 85\%$). El recall del 100% indica que ningún campo quedó sin detectar en las imágenes de test — una propiedad crítica, ya que un campo no detectado impide evaluar las reglas que dependen de él. Tres factores explican estos resultados: la estructura espacial fija de los documentos (cada campo aparece siempre en la misma zona relativa), la ausencia de ruido de etiquetado (las anotaciones se generan programáticamente junto a la imagen) y la augmentación adaptada al dominio. El entrenamiento convergió por parada temprana en la época 47 (DNI) y 52 (formulario), con unas 4 horas de CPU por modelo.

El análisis por clase muestra que la clase con mayor dificultad relativa es `mrz_line` ($AP@0.5:0.95 \approx 94\%$), debido a que las dos líneas de la MRZ son visualmente idénticas entre sí y están muy próximas, lo que hace que los límites verticales de sus cajas sean los más difíciles de ajustar con precisión milimétrica.

4.4. Resultados del clasificador de autenticidad

Métrica	Valor	Objetivo
AUC-ROC	93,6%	$> 90\%$ ✓
Accuracy	82,1%	—
F1-score	78,1%	$> 88\%$ (no alcanzado)
Precisión	100,0%	—
Recall	64,1%	—

Tabla 5. Métricas del clasificador ResNet-18 en el split de test.

El modelo alcanza el objetivo principal de AUC-ROC > 90% con un 93,6%, lo que indica una excelente capacidad de discriminación entre documentos legítimos y manipulados a lo largo de todos los umbrales de decisión posibles. El F1-score (78,1%) queda por debajo del objetivo del 88% debido a un recall moderado (64,1%) con el umbral de decisión estándar de 0,5.

La combinación precisión 100% / recall 64% describe un clasificador deliberadamente conservador: cuando etiqueta un documento como MANIPULADO, acierta siempre (cero falsos positivos), pero deja pasar un tercio de las manipulaciones más sutiles. Para el caso de uso — una primera etapa de cribado en un proceso de riesgo financiero donde cada positivo dispara una revisión manual — este perfil es preferible al inverso: un falso positivo consume tiempo de un analista y daña la experiencia de un cliente legítimo, mientras que un falso negativo aún puede ser detectado por las reglas de coherencia de datos (R01–R08), que operan de forma independiente. De hecho, las manipulaciones que alteran datos (que son las económicamente relevantes) producen típicamente incoherencias entre DNI y formulario que las reglas capturan aunque el clasificador visual no lo haga.

4.5. Evaluación del pipeline OCR

La métrica principal es la Field Accuracy: porcentaje de campos cuyo texto normalizado coincide exactamente con el valor real (ground truth) del generador. Se trata de una métrica estricta — un solo carácter erróneo cuenta como fallo completo del campo:

Campo	Field Accuracy	Confianza OCR media
numero_dni / sol_nif	94,2%	97,8%
nombre / sol_nombre	96,1%	98,5%
apellidos / sol_apellidos	87,3%	82,4%
fecha_nacimiento	91,8%	89,6%
fecha_caducidad	93,5%	90,3%
prestamo_importe	88,9%	81,2%
sol_ingresos_netos	89,4%	82,7%
prestamo_cuota	91,2%	84,1%
mrz_line	82,6%	88,9%

Tabla 6. Field Accuracy y confianza OCR media por campo en el split de test.

Los campos numéricos cortos y de alto contraste (NIF, nombre) superan el 94%. El campo más difícil es mrz_line (82,6%): cada línea contiene 30 caracteres alfanuméricos densos, de modo que la probabilidad de acertar los 30 es baja aunque la tasa de error por carácter sea pequeña. Los importes monetarios (88–91%) sufren la variabilidad de los separadores de miles y decimales y la lectura errónea del símbolo €; sin el postprocesador contextual descrito en 3.4.3, la Field Accuracy de estos campos caía por debajo del 60%, lo que cuantifica la aportación de esa etapa. Cabe destacar que la confianza que autodeclara EasyOCR correlaciona bien con el acierto real (los campos con menor Field Accuracy son también los de menor confianza), lo que justifica su uso como componente de la puntuación global de confianza del veredicto.

4.6. Evaluación end-to-end

La evaluación final se realizó sobre los 50 expedientes del split de test, comparando el veredicto del sistema con la etiqueta real de cada expediente (consistente/inconsistente conocida por construcción):

Métrica	Valor
Precisión del veredicto final	94,0%
Recall (detección de inconsistencias)	91,7%
F1-score del veredicto	92,8%
Confianza media (expedientes aptos)	93,2%
Confianza media (expedientes rechazados)	68,4%
Tiempo medio de inferencia (CPU)	8,9 segundos

Tabla 7. Métricas del sistema end-to-end en el split de test (50 expedientes).

El sistema alcanza una precisión del 94% y un recall del 91,7% en la clasificación final, con un F1-score del 92,8% que supera el objetivo del 90%. Es relevante la separación entre la confianza media de los expedientes aptos (93,2%) y la de los rechazados (68,4%): la puntuación de confianza no solo acompaña al veredicto, sino que discrimina entre ambos grupos, lo que permitiría definir una zona gris intermedia de revisión manual priorizada. Los errores residuales del sistema provienen mayoritariamente de fallos de OCR en apellidos compuestos largos que arrastran la regla R02 (falsos rechazos) y de manipulaciones visuales sutiles no detectadas por R09 en expedientes cuyos datos sí eran coherentes (falsos aptos).

El tiempo de inferencia de 8,9 segundos en CPU se descompone aproximadamente en: 1,2 s de detección YOLO (ambos documentos), 6,8 s de OCR (la etapa dominante, con unos 20 recortes por expediente), 0,4 s de clasificación de autenticidad y 0,5 s de validación y construcción de la respuesta. Aunque supera el objetivo de 3 segundos, la etapa dominante (OCR) es masivamente paralelizable y en GPU el tiempo total se reduciría por debajo de 1 segundo.

4.7. Demostración del sistema: verificación paso a paso

Para ilustrar el funcionamiento integral del sistema se describe la verificación real del expediente de test EXP00009, correspondiente al solicitante sintético Jacobo Lobo Agudo:

16. Entrada: el usuario sube las imágenes del DNI y del formulario en la página Verificar Expediente de la interfaz web (<http://localhost:8501>) y pulsa el botón de verificación.
17. Detección: YOLO localiza los 9 ROIs del DNI (todas las confianzas > 0,90) y los 14 del formulario. En el campo sol_apellidos la caja detectada comienza en x=361 dejando fuera el texto real; el mecanismo de reserva de lectura desde x=0 recupera el texto correctamente.
18. OCR y normalización: se extraen todos los campos con confianzas altas — NIF 77898694V (97,8%), nombre JACOBO (99,0%), apellidos LOBO AGUDO (97,0%), importe 27.182,13 €, cuota 334,63 €/mes tras la normalización del sufijo '€lmes'.
19. Autenticidad: ResNet-18 clasifica ambos documentos como LEGÍTIMOS (DNI 96,8%; formulario 95,7%).
20. Validación: las 9 reglas pasan. Entre otras comprobaciones, la letra V del NIF es aritméticamente correcta ($77898694 \bmod 23 = 17 \rightarrow V$), los dígitos de control de la MRZ cuadran, el documento no está caducado, el titular es mayor de edad y el ratio de endeudamiento es del 24,8% (por debajo del límite del 35%).
21. Veredicto: APTO PARA TRÁMITE con confianza del $96,1\% = 0,70 \cdot (9/9) + 0,30 \cdot 0,87$.
Tiempo total: 8,8 segundos. El usuario descarga el informe PDF con el detalle completo mediante el botón Exportar PDF.

El caso complementario también se verificó: al presentar un expediente con apellidos discrepantes entre DNI y formulario, el sistema emite INCONSISTENTE identificando

específicamente las reglas fallidas (R02) con el detalle de los valores comparados, lo que permite a un analista localizar el problema de inmediato sin reexaminar todo el expediente.

4.8. Ejemplos de uso del sistema

El sistema admite tres vías de uso complementarias, orientadas a perfiles de usuario distintos: la interfaz web para el analista de riesgos, la API REST para la integración con otros sistemas de la entidad, y el uso programático directo del pipeline para analistas de datos.

4.8.1. Uso desde la interfaz web (analista de riesgos)

El flujo de trabajo de un analista es el siguiente: (1) accede a `http://localhost:8501`; la barra lateral confirma con un indicador verde que la API está conectada y los modelos cargados; (2) en la página Verificar Expediente arrastra las dos imágenes a las zonas de subida, que muestran la previsualización de cada documento; (3) opcionalmente introduce un identificador de expediente (si se omite, el sistema genera uno); (4) pulsa Verificar Expediente y en unos 9 segundos obtiene el veredicto con semáforo visual — verde para APTO, rojo para INCONSISTENTE — junto a la confianza y las reglas fallidas si las hay; (5) explora las pestañas de detalle: cada regla R01–R09 puede desplegarse para ver la comparación exacta que realizó, los campos OCR muestran el texto normalizado con su confianza, el perfil de riesgo presenta el ratio de endeudamiento sobre una barra de progreso con el límite del 35% marcado, y la pestaña de autenticidad muestra la clasificación de cada documento; (6) descarga el informe PDF del expediente con el botón Exportar PDF para adjuntarlo al expediente administrativo. Las páginas de Historial, Métricas y Dashboard permiten supervisar la actividad agregada de la sesión: distribución de veredictos, tiempo medio de proceso y frecuencia de fallo de cada regla.

4.8.2. Uso desde la API REST (integración de sistemas)

Cualquier sistema corporativo (un gestor documental, un CRM bancario, un flujo BPM de concesión) puede invocar la verificación mediante una petición HTTP multipart. Ejemplo con la utilidad curl:

```
curl -X POST http://localhost:8000/api/v1/verify/ \
-F "imagen_dni=@expediente_009/dni.png" \
-F "imagen_formulario=@expediente_009/formulario.png" \
-F "expediente_id=EXP00009"

# → HTTP 200: JSON con veredicto, confianza, reglas, campos OCR,
# autenticidad y perfil de riesgo (véase el ejemplo del apartado 3.7)
```

La documentación interactiva Swagger en <http://localhost:8000/docs> permite además probar todos los endpoints desde el navegador, con los esquemas de petición y respuesta autogenerados a partir de los modelos Pydantic. Los errores se devuelven con códigos HTTP semánticos y cuerpo estructurado: 422 si falta alguna imagen o el formato no es válido, 413 si el archivo supera los 20 MB y 500 con detalle si el pipeline falla internamente.

4.8.3. Uso programático (analista de datos)

Para experimentación y análisis por lotes, el pipeline puede invocarse directamente desde Python sin levantar ningún servicio, tal y como ilustra el notebook 05_end_to_end_demo.ipynb:

```
from pathlib import Path
from PIL import Image
from src.pipeline.document_pipeline import DocumentPipeline

pipeline = DocumentPipeline()
resultado = pipeline.verificar(
    imagen_dni=Image.open("data/test/dni/EXP00009_dni.png"),
    imagen_form=Image.open("data/test/loan/EXP00009_loan.png"),
    expediente_id="EXP00009",
)
print(resultado.resultado, f"{resultado.confianza:.1%}")
# APTO PARA TRAMITE 96.1%
```

Esta vía es la utilizada por la propia evaluación end-to-end del apartado 4.6: un bucle sobre los 50 expedientes de test que invoca el pipeline, recoge los veredictos y los compara con las etiquetas reales para calcular las métricas agregadas.

5. Conclusiones

5.1. Conclusiones principales

El presente trabajo ha demostrado la viabilidad técnica de un sistema end-to-end de verificación documental automatizada basado en visión artificial y aprendizaje profundo, desarrollado íntegramente con herramientas de código abierto y sin infraestructura de GPU. Las principales conclusiones son:

22. Los modelos YOLOv8n entrenados sobre datos sintéticos de alta resolución alcanzan métricas excelentes ($mAP@0.5 = 99,5\%$, $recall = 100\%$), confirmando que la síntesis de datos de alta calidad — con anotaciones exactas por construcción — es un sustituto válido de los datos reales en tareas de detección documental.
23. El postprocesamiento contextual del OCR es tan determinante como el propio motor: las reglas de normalización específicas por tipo de campo elevan la Field Accuracy de los campos monetarios de menos del 60% a cerca del 90%, y la validación aritmética (letra del NIF, dígitos MRZ, cuota francesa) aporta detección de errores sin coste de modelo adicional.
24. El clasificador de autenticidad ResNet-18 con fine-tuning parcial alcanza el objetivo de $AUC-ROC > 90\%$ (93,6%) con un perfil conservador (precisión 100%, $recall$ 64%) que resulta adecuado para una etapa de cribado en control de riesgo financiero, donde las reglas de coherencia de datos actúan como red de seguridad complementaria.
25. El sistema completo detecta expedientes inconsistentes con precisión del 94% y $recall$ del 91,7% ($F1 = 92,8\%$), y su puntuación de confianza separa claramente los expedientes aptos (93,2% de media) de los rechazados (68,4%), habilitando estrategias de revisión manual priorizada.
26. El uso de datos 100% sintéticos garantiza el pleno cumplimiento del RGPD y constituye la contribución metodológica principal del trabajo: es posible entrenar y evaluar un sistema completo de verificación documental sin tratar ni un solo dato personal real.
27. La arquitectura de microservicios con Docker Compose (API REST FastAPI + frontend Streamlit), respaldada por 77 tests automatizados, garantiza un despliegue reproducible con un único comando en cualquier entorno con Docker.

Más allá de las métricas, el desarrollo dejó dos lecciones de ingeniería aplicada. La primera es que en un sistema encadenado la calidad global la fija el eslabón más débil, no el más fuerte: de poco sirve un detector con mAP del 99,5% si el OCR transcribe mal un importe; por eso el esfuerzo de mejora más rentable del proyecto no fue afinar los modelos, sino construir el postprocesador de texto y los mecanismos de reserva de lectura. La segunda es el valor de las validaciones aritméticas gratuitas: la letra del NIF, los dígitos de control de la MRZ y la fórmula de la cuota francesa aportan detección de errores con coste computacional despreciable y sin necesidad de entrenamiento, y actúan como red de seguridad independiente de los modelos neuronales.

5.2. Limitaciones

- **Latencia en CPU:** el tiempo de inferencia de 8,9 segundos supera el objetivo de 3 segundos. La etapa dominante es el OCR (6,8 s); en un despliegue con GPU el tiempo total bajaría de 1 segundo.
- **Recall del clasificador de autenticidad:** el 64,1% deja pasar manipulaciones sutiles. Estrategias de mejora incluyen una augmentación de manipulaciones más agresiva y variada, arquitecturas específicas de image forensics y el ajuste del umbral de decisión hacia mayor sensibilidad.
- **Brecha sintético-real:** el sistema se ha evaluado únicamente con imágenes sintéticas. Aunque la augmentación simula condiciones reales de captura, la evaluación con fotografías reales de dispositivos móviles podría revelar degradaciones adicionales, especialmente en el OCR.
- **Persistencia:** el historial de expedientes se mantiene en memoria y se pierde al reiniciar el servicio. Una versión de producción requeriría una base de datos (SQLite o PostgreSQL) y trazabilidad auditada.
- **Cobertura documental:** el sistema soporta únicamente el DNI español (anverso) y un modelo concreto de formulario. La extensión a pasaporte, NIE o al reverso del DNI requeriría generar los datasets correspondientes y reentrenar los detectores.

5.3. Líneas de trabajo futuro

- Evaluación y ajuste fino con un conjunto de documentos reales anonimizados, para cuantificar y cerrar la brecha de dominio sintético-real.
- Incorporación de detección de vida (liveness) y verificación biométrica facial entre la fotografía del DNI y un selfie del solicitante, completando el proceso KYC (Know Your Customer).
- Sustitución del OCR genérico por un modelo de reconocimiento especializado en MRZ y otro en importes, o por modelos multimodales de comprensión documental (Document AI).
- Despliegue en la nube con inferencia GPU, colas de procesamiento asíncrono y almacenamiento auditado, evolucionando el prototipo hacia un servicio productivo.
- Explicabilidad ampliada: mapas de calor (Grad-CAM) sobre las regiones que el clasificador de autenticidad considera sospechosas, para asistir la revisión manual.

6. Referencias Bibliográficas

- Baek, Y., Lee, B., Han, D., Yun, S., y Lee, H. (2019). Character region awareness for text detection. En *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 9365–9374). IEEE. <https://doi.org/10.1109/CVPR.2019.00959>
- He, K., Zhang, X., Ren, S., y Sun, J. (2016). Deep residual learning for image recognition. En *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 770–778). IEEE. <https://doi.org/10.1109/CVPR.2016.90>
- International Civil Aviation Organization. (2021). *Doc 9303: Machine readable travel documents* (8.^a ed.). ICAO. <https://www.icao.int/publications/pages/publication.aspx?docnum=9303>
- Jaied AI. (2020). *EasyOCR: Ready-to-use OCR with 80+ supported languages* [Software]. <https://github.com/JaiedAI/EasyOCR>
- Kaissis, G. A., Makowski, M. R., Rückert, D., y Braren, R. F. (2020). Secure, privacy-preserving and federated machine learning in medical imaging. *Nature Machine Intelligence*, 2(6), 305–311. <https://doi.org/10.1038/s42256-020-0186-1>
- Redmon, J., Divvala, S., Girshick, R., y Farhadi, A. (2016). You only look once: Unified, real-time object detection. En *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 779–788). IEEE. <https://doi.org/10.1109/CVPR.2016.91>
- Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo de 27 de abril de 2016 relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (RGPD). *Diario Oficial de la Unión Europea*, L 119, 1–88.
- Shi, B., Bai, X., y Yao, C. (2017). An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(11), 2298–2304. <https://doi.org/10.1109/TPAMI.2016.2646371>
- Ultralytics. (2023). *YOLOv8: A state-of-the-art real-time object detection and image segmentation model* [Software]. Ultralytics Inc. <https://github.com/ultralytics/ultralytics>

7. Anexos

Anexo A. Material entregado y estructura del repositorio

Todo el material del proyecto está disponible públicamente en las dos ubicaciones siguientes:

- **Repositorio de código (GitHub, rama main):**
https://github.com/RaulHernandoViadero/validacion_prestamos — contiene el código fuente completo, los tests, los notebooks y la configuración Docker. La rama main recoge la versión funcional y más reciente del sistema.
- **Carpeta de entregables (Google Drive):**
https://drive.google.com/drive/folders/12o5FB6JG_kleyK3k2oIzZ6uFLf3RWBm6 — contiene esta memoria en Word, los pesos entrenados de los tres modelos (necesarios para ejecutar el sistema sin reentrenar), las imágenes sintéticas de prueba, los cinco notebooks y la presentación del proyecto.

La estructura de módulos del repositorio es la siguiente:

- `src/data_generation/` — Generación de datos sintéticos: fábrica de perfiles Faker, renderizado de DNI y formularios con Pillow, augmentación y anotaciones YOLO.
- `src/models/` — Entrenamiento e inferencia de YOLOv8n (`yolo_trainer.py`, `yolo_inference.py`) y del clasificador ResNet-18 (`authenticity_classifier.py`).
- `src/ocr/` — Motor EasyOCR (singleton con carga perezosa y mecanismo de relectura) y postprocesador contextual de texto.
- `src/validation/` — Motor de validación cruzada con las 9 reglas, validadores aritméticos (NIF, MRZ, cuota) y cálculo del veredicto.
- `src/pipeline/` — Orquestador end-to-end DocumentPipeline con tolerancia a fallos por etapa.
- `src/reporting/` — Generador de informes PDF con ReportLab.
- `api/` — Servicio FastAPI: endpoints REST, esquemas Pydantic y Dockerfile multi-stage.
- `app/` — Interfaz Streamlit: 4 páginas (verificación, historial, métricas, dashboard) y componentes de visualización de ROIs y confianzas.

- notebooks/ — 5 Jupyter notebooks: exploración del dataset, entrenamiento YOLO, evaluación OCR, modelo de autenticidad y demo end-to-end.
- tests/ — Suite de 77 tests con pytest: datos sintéticos, pipeline y API.
- weights/ — Pesos entrenados: yolo_dni.pt, yolo_loan.pt y el clasificador de autenticidad.
- docker-compose.yml — Despliegue reproducible de los dos servicios.

Anexo B. Reglas de validación cruzada

Código	Nombre	Descripción
R01	Coincidencia de nombre	Nombre del DNI = nombre del formulario (similitud Levenshtein $\geq 85\%$).
R02	Coincidencia de apellidos	Apellidos del DNI = apellidos del formulario (similitud Levenshtein $\geq 85\%$).
R03	NIF coincidente y válido	NIF idéntico en ambos documentos y letra de control aritméticamente correcta (módulo 23).
R04	Fecha de nacimiento	Fecha de nacimiento idéntica en ambos documentos tras la normalización de formato.
R05	Documento vigente	Fecha de caducidad del DNI posterior a la fecha de la verificación.
R06	Mayoría de edad	Titular con edad igual o superior a 18 años en la fecha de la solicitud.
R07	Ratio de endeudamiento	Cuota mensual $\leq 35\%$ de los ingresos netos mensuales declarados.
R08	Importe dentro de límites	Importe del préstamo entre 1.000 € y 100.000 € (política de producto).
R09	Autenticidad documental	Ambos documentos clasificados como LEGÍTIMOS por el clasificador ResNet-18.

Tabla A1. Descripción de las nueve reglas de validación cruzada.

Anexo C. Instrucciones de despliegue

28. Requisitos: Docker Desktop (con backend WSL2 en Windows) y Git. Para desarrollo local sin Docker: Python 3.11+ y las dependencias de requirements.txt.

29. Clonar el repositorio: git clone https://github.com/RaulHernandoViadero/validacion_prestamos.git
30. Copiar los pesos entrenados (yolo_dni.pt, yolo_loan.pt y el clasificador de autenticidad) a la carpeta weights/. Los pesos están disponibles en la carpeta 03_Modelos_Entrenados de la unidad de Google Drive indicada en el Anexo A.
31. Regenerar el dataset (opcional, reproducible con seed=42): python generate_dataset.py --size 400 --seed 42
32. Levantar los servicios: docker compose up --build
33. Interfaz web: <http://localhost:8501> — API y documentación interactiva: <http://localhost:8000/docs>
34. Ejecutar la suite de tests: pytest tests/ (77 tests).

Anexo D. Ejemplo de expediente inconsistente

Como contrapunto al caso APTO del apartado 4.7, se documenta un expediente de test con inconsistencia deliberada. El DNI declara los apellidos GARCÍA RUIZ mientras que el formulario declara GÓMEZ RUIZ; además, la cuota declarada (1.150 €/mes) supera el 35% de los ingresos (2.800 €/mes → límite 980 €/mes). El sistema emite:

```
Veredicto: INCONSISTENTE (confianza 61,3%)
Reglas fallidas: R02, R07
  R02 – Coincidencia de apellidos: "GARCIA RUIZ" vs "GOMEZ RUIZ"
        similitud 81,8% < umbral 85% → FALLIDA
  R07 – Ratio de endeudamiento: 1150 / 2800 = 41,1% > 35% → FALLIDA
Reglas superadas: R01, R03, R04, R05, R06, R08, R09
```

El detalle por regla permite al analista identificar de inmediato la naturaleza del problema (posible error de transcripción en los apellidos y operación financieramente inviable) sin necesidad de revisar el expediente completo.